МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ПРИРОДНИЧО-ГУМАНІТАРНИЙ ФАХОВИЙ КОЛЕДЖ
ДВНЗ «УЖГОРОДСЬКИЙ УНІВЕРСИТЕТ»

«РОЗРОБКА WEB-ЗАСТОСУВАНЬ»

ЗВІТ З ПРАКТИЧНОЇ РОБОТИ № 5

Модульна архітектура та ін’єкція залежностей у NestJS

Виконав: Ткаченко В. М., студент курсу КН-41

Перевірив: викл. Верещагін О. О.

УЖГОРОД – 2026

Тема, мета і постановка задачі

Тема: модульна архітектура та ін’єкція залежностей у NestJS.

Мета: розділити серверний застосунок на функціональні та спільний модулі; опанувати імпорт і експорт провайдерів, стратегії useValue та useFactory, хук OnModuleInit і область видимості Scope.REQUEST.

Варіант 3: інтернет-магазин одягу та взуття. Основний ресурс — clothes; модуль, сутність і сервіс — ClothesModule, ClothingEntity, ClothesService.

Структура застосунку

Модуль	Призначення	Зв’язки
ClothesModule	CRUD каталогу; початкові товари; генерація slug	Імпортує SharedModule; експортує ClothesService
OrdersModule	Створення й перегляд замовлень; розрахунок вартості	Імпортує ClothesModule і SharedModule
SharedModule	SlugService, конфігурація, податок, request ID	Експортує сервіси і токени

AppModule містить лише імпорти трьох модулів. Контролери і сервіси ресурсів зареєстровано у власних модулях. OrdersModule отримує доступ до каталогу лише через експортований ClothesService — це демонструє інкапсуляцію провайдерів.

Середовище: Node.js 22.23.3, npm 12.2.0, TypeScript, NestJS. Сховище даних — in-memory; після перезапуску застосунку воно очищується.

Хід роботи

Рівень 1. Ізоляція каталогу та спільний модуль

• Створено ClothesModule з ClothesController і ClothesService. Реалізовано маршрути GET, POST, PATCH, DELETE та DTO з перевіркою вхідних даних.

• SlugService транслітерує кирилицю, прибирає сторонні символи та формує slug у нижньому регістрі. Наприклад: «Черевики Trail 2» → «cherevyky-trail-2».

• ClothesService впроваджує SlugService через конструктор та записує slug під час створення або перейменування товару.

Рівень 2. Замовлення та кастомні провайдери

• OrdersModule імпортує ClothesModule і викликає ClothesService.findOne(productId). За відсутності товару повертається 404 Not Found.

• STORE_CONFIG створено через useValue: назва магазину, валюта UAH, податок 20%. FEE_CALCULATOR створено через useFactory з inject: [STORE_CONFIG].

• OrdersService отримує токени через @Inject(), обчислює subtotal, taxAmount і total, зберігає замовлення у пам’яті.

Приклад перевіреного розрахунку: 1499,50 × 2 = 2999,00 грн; податок 20% = 599,80 грн; усього = 3598,80 грн.

Рівень 3. Життєвий цикл, scope і тестування

• OnModuleInit каталогу додає два початкові товари й повідомляє про ініціалізацію. OrdersService повідомляє про готовність приймати замовлення.

• RequestIdService має Scope.REQUEST. Заголовок X-Request-Id змінюється між запитами, а X-Request-Trace-Match: true підтверджує спільний екземпляр у контролері та сервісі в межах одного запиту.

• Створено requests.http і наскрізний HTTP-тест для позитивних та негативних сценаріїв. Застосунок успішно збирається; тест завершується без помилок.

Результати запуску та каталогу

Запуск NestJS: усі модулі ініціалізовано

```
[LOG] [NestFactory] Starting Nest application...
[LOG] [InstanceLoader] AppModule dependencies initialized
[LOG] [InstanceLoader] SharedModule dependencies initialized
[LOG] [InstanceLoader] ClothesModule dependencies initialized
[LOG] [InstanceLoader] OrdersModule dependencies initialized
[LOG] [RoutesResolver] ClothesController {/clothes}:
[LOG] [RouterExplorer] Mapped {/clothes, POST} route
[LOG] [RouterExplorer] Mapped {/clothes, GET} route
[LOG] [RouterExplorer] Mapped {/clothes/:id, GET} route
[LOG] [RouterExplorer] Mapped {/clothes/:id, PATCH} route
[LOG] [RouterExplorer] Mapped {/clothes/:id, DELETE} route
[LOG] [RoutesResolver] OrdersController {/orders}:
[LOG] [RouterExplorer] Mapped {/orders, POST} route
[LOG] [RouterExplorer] Mapped {/orders, GET} route
[LOG] [ClothesService] Ініціалізація модуля завершена. Завантажено початкові тестові дані.
[LOG] [OrdersService] Сервіс замовлень готовий до прийому замовлень.
[LOG] [NestApplication] Nest application successfully started
```

Рис. 1 – Успішний запуск AppModule, SharedModule, ClothesModule й OrdersModule; помилок немає.

Початковий каталог: OnModuleInit

GET /clothes — 200 OK

Відповідь сервера


```
[
  {
    "id": "clothing-1",
    "title": "Куртка зимова",
    "slug": "kurtka-zymova",
    "category": "Одяг",
    "price": 2500
  },
  {
    "id": "clothing-2",
    "title": "Кросівки Sport",
    "slug": "krosivky-sport",
    "category": "Взуття",
    "price": 1800
  }
]
```


Заголовки

```
X-Request-Id: 2d6be561-d7bc-4b9c-b9cc-cc0c9e2273da
X-Request-Trace-Match: true
```

Рис. 2 – GET /clothes: два початкові товари, додані через OnModuleInit.

Створення товару й замовлення

Створення товару та slug

POST /clothes — 201 Created

Тіло запиту

```
{
  "title": "Черевики Trail 2",
  "category": "Взуття",
  "price": 1499.5
}
```

Відповідь сервера

```
{
  "id": "clothing-3",
  "title": "Черевики Trail 2",
  "slug": "cherevyky-trail-2",
  "category": "Взуття",
  "price": 1499.5
}
```

Заголовки

```
X-Request-Id: d9bd7e52-0a09-4d8a-8709-532ec23169e9
X-Request-Trace-Match: true
```

Рис. 3 – POST /clothes: статус 201 Created і автоматично створений slug.

Замовлення: ціна, податок, підсумок

POST /orders — 201 Created

Тіло запиту

```
{
  "productId": "clothing-3",
  "quantity": 2,
  "customerEmail": "student@example.com"
}
```

Відповідь сервера

```
{
  "orderId": "ord-09428480-1096-4421-b98d-fe9ce7f17717",
  "customerEmail": "student@example.com",
  "productId": "clothing-3",
  "productTitle": "Черевики Trail 2",
  "productSlug": "cherevyky-trail-2",
  "unitPrice": 1499.5,
  "quantity": 2,
  "subtotal": 2999,
  "taxAmount": 599.8,
  "total": 3598.8,
  "currency": "UAH",
  "status": "confirmed",
  "createdAt": "2026-10-22T14:04:11.893Z"
}
```

Заголовки

```
X-Request-Id: 2253d818-a658-4255-a466-fea4b13110c2
X-Request-Trace-Match: true
```

Рис. 4 – POST /orders: актуальна ціна, кількість, податок і загальна сума.

Обробка помилки та список замовлень

Перевірка помилки 404

POST /orders — 404 Not Found

Тіло запиту

```
{
  "productId": "missing",
  "quantity": 1,
  "customerEmail": "student@example.com"
}
```

Відповідь сервера

```
{
  "message": "Товар missing не знайдено",
  "error": "Not Found",
  "statusCode": 404
}
```

Заголовки

```
X-Request-Id: b2358234-56c6-4e5e-ad2a-b50d975c7a51
X-Request-Trace-Match: true
```

Рис. 5 – POST /orders з невідомим productId повертає 404 Not Found.

Список оформлених замовлень

GET /orders — 200 OK

Відповідь сервера


```
[
  {
    "orderId": "ord-09428480-1096-4421-b98d-fe9ce7f17717",
    "customerEmail": "student@example.com",
    "productId": "clothing-3",
    "productTitle": "Черевики Trail 2",
    "productSlug": "cherevyky-trail-2",
    "unitPrice": 1499.5,
    "quantity": 2,
    "subtotal": 2999,
    "taxAmount": 599.8,
    "total": 3598.8,
    "currency": "UAH",
    "status": "confirmed",
    "createdAt": "2026-10-22T14:04:11.893Z"
  }
]
```


Заголовки

```
X-Request-Id: 7d71acda-693c-4720-9fca-04a8bc7ea4a2
X-Request-Trace-Match: true
```

Рис. 6 – GET /orders повертає збережене замовлення.

Наскрізна перевірка і область видимості

Наскрізна перевірка: npm test

```
npm notice run lab5-web-nestjs@1.0.0 test
npm notice run npm run build && node --test test/*.test.js
npm notice run lab5-web-nestjs@1.0.0 build
npm notice run tsc -p tsconfig.build.json
TAP version 13
# Subtest: повний сценарій каталогу, замовлень і request scope
ok 1 - повний сценарій каталогу, замовлень і request scope
  ---
  duration_ms: 471.7987
  type: 'test'
  ...
1..1
# tests 1
# suites 0
# pass 1
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 1902.623
```

Рис. 7 – Збірка TypeScript і наскрізний тест: 1 пройшов, 0 помилок.

Порівняння фактичних заголовків X-Request-Id у збережених результатах показало різні UUID для послідовних HTTP-запитів. Значення X-Request-Trace-Match дорівнює true: RequestIdService і RequestTraceService бачать той самий екземпляр у межах запиту. Усі три товари залишились у каталозі після створення замовлення, оскільки дані зберігає singleton-сервіс.

Тест додатково перевірив некоректні DTO (400), пошук, часткове оновлення, видалення (204) та подальший 404 для видаленого товару.

Відповіді на контрольні питання

1. Що таке Feature Module і Shared Module?

Feature Module об’єднує контролери й провайдери однієї предметної області, наприклад каталогу або замовлень. Shared Module містить повторно використовувані сервіси та експортує їх іншим модулям.

2. Як працює інкапсуляція провайдерів?

Провайдер доступний у модулі, де його оголошено. Інший модуль може його впровадити після експорту провайдера з першого модуля та імпорту цього модуля. Без експорту NestJS не зможе розв’язати залежність і повідомить про помилку запуску.

3. Чим відрізняються useClass, useValue та useFactory?

useClass створює екземпляр класу; useValue повертає готове значення, наприклад конфігурацію; useFactory викликає функцію, яка може динамічно сформувати провайдер на основі інших залежностей.

4. Для чого потрібен @Inject()?

Декоратор явно задає токен провайдера. Він потрібен для рядкових або символьних токенів, інтерфейсів і типів, яких немає під час виконання JavaScript, тому NestJS не може визначити провайдер лише за типом параметра конструктора.

Контрольні питання (продовження)

5. Як працює масив inject у useFactory?

NestJS знаходить провайдери за токенами в масиві inject і передає їх значення до фабричної функції в тому самому порядку. У проєкті STORE_CONFIG є аргументом фабрики FEE_CALCULATOR.

6. Що таке життєвий цикл та коли працює OnModuleInit?

Життєвий цикл охоплює створення, ініціалізацію та завершення компонентів. OnModuleInit викликається після розв'язання залежностей модуля. У ньому доречні початкове завантаження даних, підготовка кешу й коротка діагностика готовності.

7. Чим відрізняються Scope.DEFAULT, REQUEST і TRANSIENT?

DEFAULT зазвичай створює один екземпляр провайдера на час роботи застосунку. REQUEST створює окремий екземпляр для кожного запиту й повторно використовує його в тому самому контексті. TRANSIENT створює окремий екземпляр для кожного споживача провайдера.

8. Що таке циклічна залежність і як допомагає forwardRef()?

Цикл виникає, коли модуль або провайдер A залежить від B, а B — від A. forwardRef(() => B) відкладає розв'язання посилання до часу, коли обидві сторони відомі контейнеру NestJS. Краще спочатку перевірити архітектуру і, за можливості, винести спільну логіку в окремий модуль.

Висновок

Створено модульний REST API для магазину одягу й взуття. Реалізовано міжмодульне впровадження залежностей, кастомні провайдери конфігурації та податку, OnModuleInit і Scope.REQUEST. Фактичні HTTP-запити підтвердили коректне створення товарів і замовлень, обробку помилок та збереження стану між запитами. Наскрізний тест пройшов без помилок.

Джерела: умова практичної роботи №5; документація NestJS (docs.nestjs.com).